

Cel lekcji:

Nie projektuję tabel od nazw narzędzi.  
Projektuję model od pytania biznesowego, grainu, odbiorcy i  
definicji metryki.

Modelowanie danych to decyzja, jak przygotować dane, żeby ktoś mógł im zaufać.

```
raw/source data  -> dane przyszły z systemu  
Silver           -> dane są oczyszczone i mają stabilny grain  
fact/dim         -> dane są ułożone pod analizę  
Gold             -> dane odpowiadają na konkretne pytanie odbiorcy
```

Bez modelu każdy analityk sam robi joiny i sam definiuje metryki.

Wtedy **revenue** w jednym raporcie może znaczyć coś innego niż **revenue** w drugim.

## Przykład Danych: Zamówienia

W labie masz małe dane, które da się zobaczyć w SQLite.

```
raw_orders
order_id  customer_id  order_date  status
100       1            2026-01-01  paid
102       2            2026-01-02  cancelled
104       4            2026-01-04  refunded
```

Już tutaj jest decyzja modelowania:

Czy cancelled/refunded wchodzi do paid revenue? Nie.

## Przykład Danych: Pozycje Zamówień

Revenue nie jest zapisane gotowe w `raw_orders`.

Revenue trzeba policzyć z pozycji zamówienia:

```
raw_order_items
item_id  order_id  product_id  qty  price
1         100       10         1   120.0
2         100       11         2    80.0
```

```
line_revenue = quantity * unit_price
order 100 revenue = 1 * 120 + 2 * 80 = 280
```

To jest przykład, który można uruchomić i sprawdzić.

## Problem Bez Modelu

Source tables są zwykle projektowane pod aplikację, nie pod analitykę.

Mogą mieć:

- grain dobry dla transakcji, ale trudny dla raportów,
- statusy, które trzeba interpretować,
- wiele tabel wymagających joinów,
- nazwy i typy dobre dla systemu źródłowego,
- brak jednej oficjalnej definicji metryki.

Model danych ma zmniejszyć liczbę miejsc, w których można się pomylić.

## Trochę Historii

To są tylko dwie szkoły myślenia o porządkowaniu danych w firmie.

Nie musisz znać biografii. Masz zapamiętać intuicję:

Inmon -> najpierw wspólny, centralny model danych firmy  
Kimball -> najpierw praktyczne modele pod analitykę: fakty i  
wymiary

W tej lekcji używamy podejścia Kimballa praktycznie, bo dobrze pasuje do pytań typu:

Ile sprzedaliśmy?  
Kiedy?  
W jakim regionie?  
W jakiej kategorii produktu?

## Kimball W Jednym Zdaniu

Kimball modeling to sposób budowania analitycznych tabel wokół faktów i wymiarów.

```
fact table      = zdarzenia i liczby do agregacji  
dimension table = kontekst do filtrowania i grupowania
```

W naszym łabie pytanie brzmi:

```
Ile paid revenue mamy po dniu, regionie i kategorii produktu?
```

Dlatego rozdzielamy metrykę od opisu:

```
fact_sales: jeden rekord = jedna pozycja opłaconego zamówienia  
dim_customer: jeden rekord = jeden klient  
dim_product: jeden rekord = jeden produkt
```

Czyli:

```
fact_sales trzyma: quantity, unit_price, line_revenue  
dim_customer trzyma: customer_name, region  
dim_product trzyma: product_name, category
```

## Fact Na Konkretnym Wierszu

Z raw danych:

```
order_id 100 ma dwie pozycje:  
Keyboard: quantity 1, unit_price 120  
Mouse:    quantity 2, unit_price 80
```

W `fact_sales` to powinny być dwa rekordy:

```
fact_sales  
item_id  order_id  cust_id  product_id  date       qty  revenue  
1         100      1         10         2026-01-01  1    120.0  
2         100      1         11         2026-01-01  2    160.0
```

Dlaczego dwa? Bo grain factu to jedna pozycja opłaconego zamówienia, nie całe zamówienie.

`order_id` i `customer_id` są kluczami do połączenia z kontekstem.

Sama nazwa klienta i kategoria produktu nie muszą być skopiowane do factu.

## Dimension Na Konkretnym Wierszu

`dim_customer` nie liczy revenue. Ona opisuje klienta.

| customer_id | customer_name | region |
|-------------|---------------|--------|
| 1           | Anna Kowalska | north  |
| 2           | Bartek Nowak  | south  |

`dim_product` opisuje produkt.

| product_id | product_name | category    |
|------------|--------------|-------------|
| 10         | Keyboard     | accessories |
| 11         | Mouse        | accessories |
| 12         | Monitor      | hardware    |

Jeśli dopiszesz `total_revenue` do `dim_customer`, mieszasz opis klienta z metryką sprzedaży.

Poprawny ruch: trzymaj revenue w `fact_sales`, a region i kategorię dobieraj joinem do dimensions.

## Gdy Dimension Się Zmienia

Dimension table opisuje kontekst. Ten kontekst czasem się zmienia.

Przykłady:

```
customer_id 1 zmienia region: north -> central  
product_id 12 zmienia kategorię: hardware -> monitors
```

To jest pytanie modelowania:

```
Czy raport historyczny ma pokazać stary kontekst z dnia sprzedaży?  
Czy ma pokazać aktualny kontekst z dzisiaj?
```

## Decyzja 0 Historii Wymiaru

W głównej lekcji nie implementujemy historii wymiarów.

Masz tylko umieć zauważyć decyzję:

Jeśli raport ma pokazywać aktualny opis klienta,  
możesz nadpisać region.

Jeśli raport ma pokazywać kontekst z dnia sprzedaży,  
potrzebujesz historii wymiaru.

To jest wystarczające na core modelowania: rozumiesz konsekwencję dla revenue.

## Gdzie Jest Pełne SCD

Pełne nazwy i implementacja są w bonusie 03b.

Tam dopiero wchodzi:

```
SCD Type 1 / Type 2  
business key vs surrogate key  
valid_from / valid_to  
join historyczny po dacie
```

W lekcji 03 najpierw budujemy intuicję: dimension może się zmieniać.

Model musi więc świadomie wybrać, czy zachowuje historię.

## Fact + Dimensions Razem

Jeden wiersz `fact_sales` mówi, że coś sprzedano:

```
order_item_id  customer_id  product_id  line_revenue
1              1           10           120.0
```

Dimensions tłumaczą klucze na kontekst:

```
customer_id 1 -> Anna Kowalska, region north
product_id 10 -> Keyboard, category accessories
```

Po joinie analityk może zapytać:

```
Ile revenue było w regionie north dla kategorii accessories?
```

Wynik pochodzi z factu, a opis pochodzi z dimensions.

## Grain Jest Decyzją

Przed nazwą tabeli zapisz:

```
jeden rekord = ...
```

Przykład:

```
raw_orders: jeden rekord = jedno zamówienie  
raw_order_items: jeden rekord = jedna pozycja zamówienia  
fact_sales: jeden rekord = jedna pozycja opłaconego zamówienia  
gold_daily_sales: jeden rekord = jeden dzień sprzedaży
```

Jeśli nie znasz grainu, nie wiesz jeszcze, co modelujesz.

## Bronze, Silver, Gold

Medallion architecture to praktyczny sposób porządkowania zaufania do danych.

```
Bronze = blisko źródła  
Silver = oczyszczone, ujednolicone, sprawdzone  
Gold   = gotowe pod odbiorcę albo use case
```

To nie są magiczne foldery. To pytanie: co już wiemy o danych i komu wolno ich używać?

## Jak To Się Łączy

Te pojęcia nie konkurują ze sobą. One odpowiadają na różne pytania.

```
Medallion -> na jakim poziomie zaufania są dane?  
Kimball   -> jak ułożyć dane analitycznie?  
Star      -> jak wygląda model fact + dimensions?  
Gold      -> jaka gotowa tabela pomaga odbiorcy?  
OLAP      -> jak szybko pytać i agregować dane?
```

Najprostsza mapa:

```
raw/Bronze -> Silver -> Kimball fact/dim -> Gold tables ->  
OLAP/dashboard
```

## Ten Sam Order W Różnych Warstwach

Nie robimy kilku wersji danych bez powodu.

Każda warstwa odpowiada na inne pytanie o ten sam `order_id = 100`:

```
raw_orders
  Co przyszło z systemu?
  order 100, status paid, data 2026-01-01

silver_orders
  Czy zapis jest oczyszczony i spójny?
  status = paid, data ma poprawny format

fact_sales
  Co faktycznie sprzedaliśmy?
  order 100 ma dwie pozycje sprzedaży: 120.0 i 160.0

gold_daily_sales
  Jaki wynik pokaże dashboard?
  dzień 2026-01-01 dostaje +280.0 revenue
```

Czyli: raw przechowuje źródło, Silver porządkuje, fact liczy zdarzenia sprzedaży, a Gold pokazuje gotowy wynik dla odbiorcy.

## Bronze

Bronze jest blisko source systemu.

W tej lekcji `raw` traktujemy jak lokalny odpowiednik Bronze:

```
raw_customers  
raw_products  
raw_orders  
raw_order_items
```

Bronze jest ważne, bo pozwala odtworzyć przetwarzanie, gdy zmienisz logikę Silver albo Gold.

Nie traktuj Bronze jako trusted modelu raportowego.

## Silver Na Konkretnym Przykładzie

Silver nie musi robić wielkiej magii.

Przykład prostego porządkowania:

```
raw_customers.customer_name -> TRIM(customer_name)
raw_customers.region        -> LOWER(TRIM(region))
raw_orders.status           -> LOWER(TRIM(status))
```

Czyli Silver odpowiada:

```
Czy dane mają spójny zapis?
Czy status jest znany?
Czy downstream nie musi zgadywać formatu?
```

## Silver

Silver porządkuje dane.

Typowe zadania Silver:

- jawne kolumny,
- spójny case tekstu,
- znane statusy,
- podstawowe reguły jakości,
- stabilny grain,
- nazwy, które downstream rozumie.

Silver nie musi być raportem. Silver ma być bezpiecznym wejściem do modelowania.

## Gold

Gold odpowiada na konkretne pytanie albo potrzeby odbiorcy.

Przykłady:

```
gold_daily_sales: dashboard dziennej sprzedaży  
gold_sales_by_region_category: opcjonalne rozszerzenie pod analizę  
region x kategoria
```

Gold powinien mieć odbiorcę, grain, definicje metryk, checki jakości i opis, do czego tej tabeli nie używać.

## Gold Na Konkretnym Wyniku

`gold_daily_sales` może wyglądać tak:

| order_date | orders | items | revenue |
|------------|--------|-------|---------|
| 2026-01-01 | 1      | 3     | 280.0   |
| 2026-01-02 | 1      | 1     | 900.0   |
| 2026-01-03 | 1      | 2     | 300.0   |
| 2026-01-05 | 1      | 5     | 240.0   |
| 2026-01-06 | 1      | 2     | 1800.0  |

To jest wygodne dla dashboardu dziennego.

Ale nie używaj tej tabeli do analizy per produkt, bo ona już zgubiła `product_id` i `category`.

## Gdzie Tu Jest Kimball

Kimball zwykle żyje po Silver i przed Gold.

```
raw/Bronze
-> Silver
    -> dim_customer
    -> dim_product
    -> fact_sales
    -> Gold
```

Dlaczego po Silver?

Bo fact/dim powinny dostać dane oczyszczone, nie losowe statusy i formaty z raw.

Dlaczego przed Gold?

Bo wiele Goldów może używać tej samej oficjalnej definicji revenue z fact\_sales.

## Star Schema

Schemat gwiazdy ma fakt w centrum i wymiary dookoła.

```
      dim_customer  
      |  
dim_product ---- fact_sales
```

W tej lekcji minimum to:

```
dim_customer + fact_sales + dim_product
```

Star schema jest dobra, gdy chcesz elastycznie analizować metryki po wielu przekrojach.

## Star To Kształt Kimballa

Kimball to podejście.

Star schema to typowy kształt tego podejścia.

Kimball pyta:  
co jest faktem, co jest wymiarem, jaki jest grain?

Star schema pokazuje:  
fact table w centrum, dimensions dookoła

W tej lekcji:

Kimball decision: revenue jest w fact\_sales  
Star shape: fact\_sales łączy się z dim\_customer i dim\_product

## Star Schema Jako Query

Jeśli chcesz revenue per region i category, używasz fact + dimensions:

```
fact_sales.line_revenue  
+ dim_customer.region  
+ dim_product.category
```

Wynik może wyglądać tak:

| region | category    | total_revenue |
|--------|-------------|---------------|
| north  | accessories | 280.0         |
| north  | hardware    | 900.0         |
| south  | accessories | 240.0         |
| west   | office      | 300.0         |
| west   | hardware    | 1800.0        |

To jest namacalne: revenue jest w fact, opisy są w dimensions.

## Star, Snowflake, Historia, Data Vault

Te nazwy nie są jednym typem decyzji.

Najprostsza mapa:

```
Star schema
  kształt modelu analitycznego: fact + dimensions

Snowflake schema
  wariant star: dimensions są rozbite na mniejsze tabele

Historia wymiaru
  decyzja, czy opis ma być aktualny czy historyczny

Data Vault
  osobny model integracji, historii i pochodzenia danych
```

W tej lekcji najważniejsze są: star schema, Gold i decyzja, czy wymiar potrzebuje historii.

## Star Vs Snowflake Schema

Star schema trzyma dimensions szerzej i bliżej fact table.

```
dim_product
product_id  product_name  category
10          Keyboard    accessories
```

Snowflake schema rozбивa dimension dalej:

```
dim_product
product_id  product_name  category_id
10          Keyboard    1

dim_category
category_id  category
1            accessories
```

Star jest prostszy dla analityka. Snowflake zmniejsza duplikację, ale wymaga więcej joinów.

## Kiedy Star, Kiedy Snowflake

W tej lekcji wybieramy star schema.

Powód:

Mały model, prosta analityka, mało tabel, łatwiejsze zapytania.

Snowflake rozważasz, gdy:

dimension jest duża,  
ma własne hierarchie,  
albo ten sam opis jest używany w wielu wymiarach.

Przykład hierarchii:

product -> category -> department

Na start nie komplikuj modelu bez potrzeby.

## Historia Wymiaru To Nie Data Vault

Historia wymiaru i Data Vault oba dotyczą czasu, ale rozwiązują inny problem.

Historia wymiaru  
opis klienta albo produktu w czasie

Data Vault  
pełna historia i lineage wielu źródeł danych

Przykład historii wymiaru:

`dim_customer` trzyma wersje regionu klienta

Przykład Data Vault:

Hub Customer + Satellite Customer Attributes

Historia wymiaru jest lokalną decyzją w modelu analitycznym. Data Vault to osobny sposób budowania warstwy integracyjnej.

## Data Vault W Jednym Slajdzie

Data Vault składa dane z wielu źródeł i zachowuje historię.

Minimalne klocki:

Hub

stabilny biznesowy identyfikator, np. customer\_id

Link

relacja między hubami, np. customer złożył order

Satellite

atrybuty i historia zmian, np. region od kiedy do kiedy

Data Vault odpowiada głównie na pytanie:

Jak bezpiecznie przechowywać historię i pochodzenie danych?

Star schema odpowiada na inne pytanie:

Jak wygodnie analizować metryki po wymiarach?

## Co Robimy W Tej Lekcji

Nie robimy pełnego Data Vault.

Nie robimy pełnej implementacji historii wymiarów.

Robimy:

```
Star schema
  fact_sales + dim_customer + dim_product

Wide Gold
  gotowe tabele pod dashboard

Mini-decyzja o historii
  czy zmiana regionu/kategorii wymaga historii?
```

To wystarczy, żeby dobrze wejść w modelowanie bez mieszania warstw.

## Co Jest Obok Star Schema

Star schema nie jest jedynym kształtem modelu.

Na tej lekcji wystarczy znać mapę:

3NF/normalizacja

dane rozbite na wiele tabel, częste w OLTP

Star schema

fact w centrum, dimensions dookoła

Historia wymiaru

decyzja, czy opis ma być aktualny czy historyczny

Denormalized/Wide Gold

jedna szersza tabela pod konkretny dashboard albo odbiorcę

Snowflake schema

wariant star z rozbitymi dimensions

Data Vault

osobna warstwa historii i integracji wielu źródeł

W praktyce najpierw naucz się dobrze uzasadnić star schema i Gold.

## Denormalizowany Gold

Denormalizacja to celowe połączenie danych w szerszą tabelę.

Przykład:

```
gold_sales_by_region_category
```

Taka tabela może mieć już:

```
order_date, region, category, total_revenue, sold_items_count
```

To jest wygodne dla dashboardu, bo odbiorca nie robi joinów.

Koszt: mniej elastyczności i ryzyko duplikowania logiki w wielu Goldach.

## Wide Gold Jako Gotowa Tabela

`gold_sales_by_region_category` może mieć od razu wszystko, czego potrzebuje dashboard:

| order_date | region | category    | revenue |
|------------|--------|-------------|---------|
| 2026-01-01 | north  | accessories | 280.0   |
| 2026-01-02 | north  | hardware    | 900.0   |
| 2026-01-03 | west   | office      | 300.0   |
| 2026-01-05 | south  | accessories | 240.0   |
| 2026-01-06 | west   | hardware    | 1800.0  |

Dashboard nie musi znać joinów.

Ale jeśli jutro ktoś zapyta o `product_name`, ta tabela może nie wystarczyć.

## A Gdzie Jest OLAP

OLAP to sposób używania danych do szybkiej analizy po wymiarach.

Najprostsze pytanie OLAP:

Pokaż revenue po dniu, regionie i kategorii.  
Pozwól filtrować, grupować, drill-down i porównywać.

OLAP nie zastępuje Medallion ani Kimballa.

Medallion porządkuje pipeline.  
Kimball/star porządkuje model analityczny.  
OLAP używa modelu do szybkich agregacji i eksploracji.

## OLAP Na Tym Samym Przykładzie

OLAP query może użyć star schema:

```
SUM(fact_sales.line_revenue)
GROUP BY dim_customer.region, dim_product.category
```

Albo może czytać gotowy Gold:

```
SELECT * FROM gold_sales_by_region_category
```

Różnica:

```
Star schema -> więcej elastyczności dla analityka.
Gold         -> mniej pracy dla dashboardu.
```

## Star Schema Vs Wide Gold

Star schema:

- + elastyczna analiza
- + mniej duplikacji kontekstu
- + dobra warstwa shared analytics
- odbiorca musi rozumieć joiny

Denormalized Gold:

- + prosty dashboard
- + mniej joinów dla odbiorcy
- + szybkie użycie
- łatwo skopiować logikę metryki w wiele miejsc

Nie ma jednej zawsze dobrej odpowiedzi. Jest use case.

## Netflix-Style Case

Wyobraź sobie Netflix albo platformę streamingową.

Najpierw konkretne zdarzenie, które da się wyobrazić:

```
watch_event
user_profile:    profile_7
title:           Stranger Things
event_time:      2026-01-01 20:01
seconds_watched: 600
device:          tv
```

To jest event. Fact może mieć grain jednego zdarzenia oglądania:

```
fact_watch_event: jeden rekord = jedno zdarzenie oglądania
```

Dimensions dodają opis:

```
dim_title: tytuł, gatunek, rok produkcji
dim_profile: kraj profilu, plan subskrypcji
dim_device: typ urządzenia
```

Gold pod dashboard może być już dzienny:

```
gold_daily_title_watch_time: jeden rekord = dzień + tytuł
```

## Dlaczego Nie Jedna Tabela Do Wszystkiego

Jedna szeroka tabela może być dobra dla jednego dashboardu.

Ale streaming ma wiele pytań:

- ranking tytułów: które tytuły mają najwięcej minut oglądania,
- usage per device: czy ludzie oglądają więcej na TV czy mobile,
- content per country: które tytuły działają w danym kraju,
- completion rate: jaki procent filmu/odcinka użytkownik obejrzał.

Jedna tabela szybko miesza różne grainy.

Core fact/dim pomaga utrzymać spójność, a Gold buduje się pod konkretne pytania.

1. Use case: kto używa danych i po co?
2. Grain: jeden rekord wyniku = co?
3. Source: które tabele wejściowe są potrzebne?
4. Silver: co trzeba oczyścić i ujednolicić?
5. Fact/dim: co jest faktem, a co kontekstem?
6. Historia: czy zmiany w wymiarach wymagają wersjonowania?
7. Gold: jaka tabela jest wygodna dla odbiorcy?
8. Checks: jak udowodnić, że model nie kłamie?

## Przykład Z Lekcji

Pytanie:

Ile paid revenue mamy dziennie?

Decyzje:

```
fact_sales grain = jedna pozycja opłaconego zamówienia
revenue = quantity * unit_price
status = paid
dim_customer daje region
dim_product daje category
gold_daily_sales grain = jeden dzień
```

Analiza region/category jest dobrym rozszerzeniem, ale podstawowy model lekcji zaczyna się od dziennego Golda.

## Najłatwiejsze Błędy

- Budowanie Gold bezpośrednio z raw i bez checków.
- Brak jednego zdania o grainie tabeli.
- Revenue jako `SUM(unit_price)` zamiast `SUM(quantity * unit_price)`.
- Dodanie `revenue` do `dim_customer`.
- Brak decyzji, czy zmiana regionu/kategorii wymaga historii.
- Zrobienie jednej wielkiej tabeli dla wszystkich pytań.
- Uznanie, że `Bronze/Silver/Gold` to tylko nazwy folderów.

## Mini-Checklist

- Czy każda tabela ma **jeden rekord = ..**?
- Czy wiadomo, kto używa Gold?
- Czy fact trzyma liczby i zdarzenia?
- Czy dimensions trzymają opis kontekstu?
- Czy revenue ma jedną definicję?
- Czy wiadomo, co robimy, gdy dimension zmienia się w czasie?
- Czy checki porównują fact z Gold?
- Czy wiesz, kiedy wybrać star schema, a kiedy wide Gold?

Bronze/Silver/Gold mówi o zaufaniu i celu warstwy.  
Kimball mówi, jak modelować analitycznie przez fact i dimensions.  
Star schema to typowy kształt Kimballa: fact w centrum, dimensions dookoła.  
Historia wymiaru mówi, co robisz, gdy dimension zmienia się w czasie.  
Denormalized Gold daje prostotę dla konkretnego dashboardu albo odbiorcy.  
OLAP to sposób szybkiego pytania i agregowania danych po wymiarach.  
Grain decyduje, czy metryka jest poprawna.